Fiche de cours PHP : variables, portée et types

Lycée Rafèha, 4ème SI. Enseignant : Aymen Essouyah

1. Déclarer une variable

En PHP, une variable commence par le signe `$`. Il n'y a **pas de mot-clé** de déclaration (ni `var`, ni `let`, ni `const`) : la variable naît à sa première affectation.

```
<?php
$nom = "Ali";
$age = 16;
$moyenne = 13.5;
```

Règles à retenir :

- Le nom commence par une lettre ou `_`, jamais par un chiffre : `$note1` est correct, `$1note` est une erreur.
- PHP distingue majuscules et minuscules : `$age` et `$Age` sont deux variables différentes.
- Le type est attaché à la **valeur** (typage dynamique), comme en JavaScript.

```
$x = 10;     // int
$x = "dix";  // string (autorisé)
```

Afficher une variable : guillemets doubles ou simples.

```
$nom = "Ali";
echo "Bonjour $nom";    // Bonjour Ali (la variable est remplacée)
echo 'Bonjour $nom';    // Bonjour $nom (texte tel quel)
echo "Bonjour " . $nom; // concaténation avec le point
```

Tester une variable : `isset($x)` (existe et n'est pas `null`), `unset($x)` (supprime), `empty($x)` (vide ou égale à zéro).

2. Les constantes

Une constante ne change jamais de valeur. Son nom ne porte **pas** de `$`.

```
define("TVA", 0.19);
const PI = 3.14;
echo TVA;          // 0.19
TVA = 0.20;        // erreur : on ne modifie pas une constante
```

Une constante est visible **partout**, y compris dans les fonctions.

3. La portée (scope)

Portée : c'est la zone du programme où une variable existe et où l'on peut l'utiliser. En dehors de cette zone, PHP ne la connaît pas.

Analogie du lycée : la portée, c'est la distance que la voix peut atteindre.

- Une annonce au haut-parleur est entendue dans le couloir et dans la cour : variable **globale**.
- Une fonction est une salle de classe **porte fermée** : l'annonce du haut-parleur n'y entre pas. Ce qui se dit dans la salle reste dans la salle : variable **locale**.
- Le mot `global` revient à ouvrir la porte pour laisser entrer l'annonce.
- Les carnets de liaison (`$_GET`, `$_POST`, `$_SESSION`) sont donnés à tous les élèves : ce sont des variables **superglobales**, entendues partout.

Variable globale et variable locale

Différence importante avec JavaScript : dans une fonction PHP, une variable globale n'est **pas** accessible par défaut.

```
<?php
$site = "STI";

function afficher() {
  echo $site;   // rien affiché : avertissement « Undefined variable $site »
}

function afficher2() {
  global $site; // on ouvre la porte
  echo $site;   // STI
}
```

Autre solution, souvent préférable : passer la valeur en **paramètre**.

```
function afficher3($s) {
  echo $s;
}
afficher3($site); // STI
```

Pas de portée de bloc

En PHP, `if`, `for` et `while` ne créent **pas** de zone propre : une variable créée dans un bloc reste visible après.

```
if (true) {
  $a = 1;
}
echo $a; // 1

for ($i = 0; $i < 3; $i++) {
  $dernier = $i;
}
echo $i;       // 3
echo $dernier; // 2
```

Seule la **fonction** crée une portée locale.

Variable statique

Une variable locale disparaît à la fin de la fonction. Avec `static`, elle **garde sa valeur** d'un appel à l'autre.

```
function compteur() {
  static $n = 0;
  $n++;
  return $n;
}
echo compteur(); // 1
echo compteur(); // 2
echo compteur(); // 3
```

4. Le hoisting (remontée)

Hoisting (remontée) : en JavaScript, les déclarations « remontent » en haut de leur zone avant l'exécution du code. **PHP ne le fait pas pour les variables** : le fichier est lu de haut en bas.

Analogie de la classe : le surveillant n'appelle que les élèves déjà inscrits et arrivés. Un élève qui n'existe pas encore sur la liste ne répond pas.

```
echo $x;   // rien affiché : avertissement « Undefined variable $x »
$x = 5;
echo $x;   // 5
```

Excepté pour les **fonctions** : elles sont connues avant le début de la lecture, comme le règlement du lycée affiché avant la rentrée.

```
echo double(4); // 8 (appel avant la définition : autorisé)

function double($n) {
  return $n * 2;
}
```

5. Les types de données

Catégorie	Types
Scalaires	`int`, `float`, `string`, `bool`
Composé	`array` (indexé ou associatif)
Spécial	`null`

(Les objets relèvent de la programmation orientée objet, non étudiée ici.)

Les types scalaires

Scalaire vient du latin *scala* (échelle, degré) : une valeur **simple, à un seul niveau**, qui ne contient rien d'autre. C'est l'équivalent de « primitif » en JavaScript.

Analogie de la cuisine : le sel, la farine et l'œuf sont des ingrédients de base, impossibles à décomposer en quelque chose de plus simple. Un gâteau, lui, est composé de plusieurs ingrédients : c'est un **tableau** (`array`). Deux sacs de farine identiques sont indépendants : vider l'un ne vide pas l'autre.

Type	Exemple	Remarque
`int`	`15`, `-3`	Entier
`float`	`3.14`, `2.5`	Réel (le séparateur est le point)

Type	Exemple	Remarque
`string`	`"Ali"`, `'texte'`	Chaîne de caractères
`bool`	`true`, `false`	Booléen

Les tableaux (`array`)

Un **tableau** range plusieurs valeurs sous un seul nom. Il existe **deux sortes de tableaux** en PHP : le tableau **indexé** et le tableau **associatif**. La différence tient à la façon de retrouver une case.

Tableau indexé : repéré par un numéro

Chaque case porte un **numéro** (l'indice), qui commence à 0.

Analogie du couloir : une rangée de casiers numérotés 0, 1, 2, 3… Pour retrouver un objet, on donne le numéro du casier.

```
$notes = [12, 15, 9];
echo $notes[1];       // 15
$notes[0] = 14;       // modifier une case
$notes[3] = 10;       // ajouter le casier n° 3
echo count($notes);   // 4

for ($i = 0; $i < count($notes); $i++) {
  echo $notes[$i];
}
```

Tableau associatif : repéré par un nom (clé)

Chaque case porte une **étiquette** (la clé) choisie par le programmeur. On écrit `clé => valeur` .

Analogie de la fiche de renseignements : une fiche d'élève avec des rubriques « nom », « age », « moyenne ». Pour lire une information, on donne le nom de la rubrique, pas un numéro.

```
$eleve = ["nom" => "Ali", "age" => 16, "moyenne" => 13.5];
echo $eleve["nom"];          // Ali
$eleve["moyenne"] = 14;      // modifier une rubrique
$eleve["classe"] = "4 SI 2"; // ajouter une rubrique

foreach ($eleve as $cle => $valeur) {
  echo "$cle : $valeur<br>";
}
```

`foreach` parcourt toutes les cases : à chaque tour, `$cle` reçoit l'étiquette et `$valeur` son contenu. On teste l'existence d'une clé avec `isset($eleve["nom"])`.

	Indexé	Associatif
Case repérée par	Un numéro (0, 1, 2…)	Une clé (un nom)
Analogie	Casiers numérotés	Fiche avec rubriques
Lecture	`$notes[1]`	`$eleve["nom"]`
Parcours	`for` avec `count()`	`foreach`
Exemple d'usage	Liste de notes	Fiche d'un élève

Le type `null`

`null` signifie « aucune valeur ». Une variable non définie ou affectée à `null` donne `isset($x)` égal à `false`.

Connaître le type : `gettype` et `var_dump`

```
gettype(15);      // "integer"
gettype(3.14);    // "double"  (c'est le nom PHP du type float)
gettype("Ali");   // "string"
gettype(true);    // "boolean"
gettype(null);    // "NULL"
gettype([1, 2]);  // "array"

var_dump(15);     // int(15)
var_dump("15");   // string(2) "15"
var_dump(3.14);   // float(3.14)
var_dump(true);   // bool(true)
var_dump(null);   // NULL
```

Tests pratiques : `is_int()`, `is_float()`, `is_string()`, `is_bool()`, `is_array()`, `is_null()`, `is_numeric()`.

```
is_numeric("15");  // true
is_numeric("15px"); // false
```

Copie : valeur ou référence

En PHP, **tout est copié par valeur**, tableaux compris (différence importante avec JavaScript).

```
$a = 5;
$b = $a;
$b = 10;
echo $a; // 5

$t1 = [1, 2, 3];
$t2 = $t1;
$t2[0] = 99;
echo $t1[0]; // 1  (t1 n'a pas changé : t2 est une vraie copie)
```

Analogie : copier par valeur, c'est donner une **photocopie** du document ; l'original reste intact. Copier par référence (`&`), c'est donner l'**original** lui-même : toute modification est vue des deux côtés.

```
function ajouter1($n) {
  $n = $n + 1;
}
function ajouter2(&$n) {   // le & transmet l'original
  $n = $n + 1;
}

$a = 5;
ajouter1($a);
echo $a; // 5
ajouter2($a);
echo $a; // 6
```

6. Conversions de types

Explicites (casting)

```
(int) "42";       // 42
(int) "15px";     // 15
(int) "abc";      // 0
(float) "3.5";    // 3.5
(string) 123;     // "123"
(bool) 0;         // false
intval("15");     // 15
floatval("3.5");  // 3.5
strval(12);       // "12"
```

Implicites : l'opérateur décide. Le `+` additionne, le `.` concatène.

```
"5" + 2;   // 7    (addition)
"5" . 2;   // "52" (concaténation)
true + 1;  // 2
```

Division :

```
10 / 4;         // 2.5
intdiv(10, 4);  // 2  (division entière)
10 % 3;         // 1  (reste)
```

7. ~~Valeurs vraies et fausses (falsy et truthy)~~ (hors programme, mais à connaître)

~~Sont considérées comme **fausses** : `false` , `0` , `0.0` , `""` , `"0"` , `[]` et `null` . Tout le reste est **vrai**. Attention : contrairement à JavaScript, la chaîne `"0"` est fausse en PHP.~~

8. Comparaison : `==` et `===`

```
5 == "5";          // true  (avec conversion)
5 === "5";         // false (type différent)
null == false;     // true
null === false;    // false
0.1 + 0.2 == 0.3;  // false (précision des réels)
```

Règle : utiliser `===` et `!==`.

9. À retenir

- Une variable commence par `$`, sans mot-clé de déclaration ; PHP distingue majuscules et minuscules.

- Une constante se crée avec `define()` ou `const`, sans `$`, et se voit partout.
- Dans une fonction, les variables globales ne sont **pas** visibles : utiliser un paramètre ou `global`.
- Seule la fonction crée une portée : `if`, `for` et `while` n'en créent pas.
- `static` garde la valeur d'une variable locale entre deux appels.
- Types scalaires : `int`, `float`, `string`, `bool` ; composé : `array` ; spécial : `null`.
- Un tableau est **indexé** (cases numérotées à partir de 0) ou **associatif** (cases repérées par une clé).
- Tout est copié par valeur ; le `&` transmet l'original.
- `+` additionne, `.` concatène ; préférer `===` à `==`.

Exercices

Exercice 1 : Que s'affiche-t-il ?

Pour chaque extrait, donner ce qui s'affiche (ou le problème rencontré).

```
// a)
$a = 1;
function f() {
  echo $a;
}
f();

// b)
$b = 2;
function g() {
  global $b;
  echo $b;
}
g();

// c)
if (true) {
  $c = 3;
}
echo $c;

// d)
function h() {
  static $n = 0;
  $n++;
```

```
  echo $n;
}
h();
h();
h();

// e)
echo $d;
$d = 4;

// f)
define("X", 10);
function k() {
  echo X;
}
k();
```

Exercice 2 : Corriger les erreurs

```
<?php
nom = "Ali";
$Age = 16;
echo "Bonjour " + $nom;
echo $age;
function salut() {
  return "Salut " . $nom;
}
```

Exercice 3 : gettype

Donner le résultat de chaque appel.

```
gettype(3.14);
gettype("3.14");
gettype(null);
gettype([1, 2]);
gettype(7);
gettype(false);
```

Exercice 4 : Conversions et comparaisons

```
// a)
echo "10" + 5;
// b)
echo "10" . 5;
// c)
echo (int) "15px";
// d)
echo (int) "abc";
// e)
echo 10 / 4;
// f)
echo intdiv(10, 4);
// g)
var_dump(5 == "5");
// h)
var_dump(5 === "5");
// i)
var_dump(0.1 + 0.2 == 0.3);
```

Exercice 5 : Valeur ou référence

```
function ajouter1($n) {
  $n = $n + 1;
}
function ajouter2(&$n) {
  $n = $n + 1;
}

$a = 5;
ajouter1($a);
echo $a;
ajouter2($a);
echo $a;

$t1 = [1, 2, 3];
$t2 = $t1;
$t2[0] = 99;
echo $t1[0];
```

Que s'affiche-t-il ? Expliquer chaque résultat.

Exercice 6 : Programmation

Écrire une fonction `moyenne($notes)` qui reçoit un tableau de nombres et retourne leur moyenne. Contraintes : une boucle `for`, la fonction `count()`, et aucune variable globale. Tester avec `[12, 15, 9]`.

Exercice 7 : Compteur d'appels

Écrire une fonction `visite()` qui affiche « Visite numéro N », où N augmente à chaque appel. L'appeler trois fois. Quel mot-clé faut-il utiliser ?

Exercice 8 : Tableaux indexés et associatifs

a) Que s'affiche-t-il ?

```
$jours = ["lundi", "mardi", "mercredi"];
echo $jours[1];
$jours[3] = "jeudi";
echo count($jours);

$eleve = ["nom" => "Sami", "age" => 17];
echo $eleve["nom"];
$eleve["age"] = 18;
$eleve["classe"] = "4 SI 1";
echo count($eleve);
var_dump(isset($eleve["moyenne"]));
```

b) Écrire un tableau associatif `$voiture` avec les clés `marque`, `couleur` et `annee`, puis afficher chaque rubrique sous la forme « marque : Renault » avec une boucle `foreach`.

c) Dire si chaque tableau est indexé ou associatif :

```
$a = [3, 6, 9];
$b = ["x" => 3, "y" => 6];
```

Corrigés

Corrigé de l'exercice 1

	Résultat	Explication
a)	Rien (avertissement « Undefined variable $a »)	La fonction ne voit pas la variable globale

	Résultat	Explication
b)	`2`	`global` ouvre la porte
c)	`3`	Pas de portée de bloc en PHP
d)	`123`	`static` garde la valeur entre les appels
e)	Rien (avertissement « Undefined variable $d »)	Pas de remontée des variables
f)	`10`	Une constante est visible partout

Corrigé de l'exercice 2

Erreurs : `nom` sans `$` ; `$Age` et `$age` sont deux variables différentes ; `+` au lieu de `.` pour concaténer ; `$nom` n'est pas visible dans la fonction.

```
<?php
$nom = "Ali";
$age = 16;
echo "Bonjour " . $nom;
echo $age;
function salut($n) {
  return "Salut " . $n;
}
echo salut($nom);
```

Corrigé de l'exercice 3

Appel	Résultat
`gettype(3.14)`	`"double"`
`gettype("3.14")`	`"string"`
`gettype(null)`	`"NULL"`
`gettype([1, 2])`	`"array"`
`gettype(7)`	`"integer"`
`gettype(false)`	`"boolean"`

Corrigé de l'exercice 4

	Résultat	Explication
a)	`15`	`+` additionne après conversion
b)	`105`	`.` concatène
c)	`15`	Lecture des chiffres du début
d)	`0`	Aucun chiffre au début
e)	`2.5`	Division réelle
f)	`2`	Division entière
g)	`bool(true)`	`==` convertit
h)	`bool(false)`	Types différents
i)	`bool(false)`	`0.1 + 0.2` n'est pas exactement `0.3`

Corrigé de l'exercice 5

```
5
6
1
```

- `ajouter1` reçoit une **copie** (photocopie) : `$a` reste à 5.
- `ajouter2` reçoit l'**original** grâce au `&` : `$a` devient 6.
- `$t2` est une vraie copie de `$t1` : modifier `$t2[0]` ne change pas `$t1[0]`, qui reste 1.

Corrigé de l'exercice 6

```php
<?php
function moyenne($notes) {
  $somme = 0;
  for ($i = 0; $i < count($notes); $i++) {
    $somme = $somme + $notes[$i];
  }
  return $somme / count($notes);
}

$t = [12, 15, 9];
echo moyenne($t); // 12
```

`$somme` et `$i` sont locales à la fonction : elles disparaissent à la fin de l'appel.

Corrigé de l'exercice 7

Il faut une variable **statique**.

```
<?php
function visite() {
  static $n = 0;
  $n++;
  echo "Visite numéro $n<br>";
}

visite(); // Visite numéro 1
visite(); // Visite numéro 2
visite(); // Visite numéro 3
```

Sans `static`, `$n` serait recréée à 0 à chaque appel et afficherait toujours 1.

Corrigé de l'exercice 8

a)

Instruction	Résultat	Explication
`echo $jours[1]`	`mardi`	Les indices commencent à 0
`echo count($jours)`	`4`	Le casier n° 3 a été ajouté
`echo $eleve["nom"]`	`Sami`	Lecture par la clé
`echo count($eleve)`	`3`	Trois rubriques : nom, age, classe
`var_dump(isset($eleve["moyenne"]))`	`bool(false)`	La clé `moyenne` n'existe pas

b)

```
$voiture = ["marque" => "Renault", "couleur" => "rouge", "annee" => 2020];
foreach ($voiture as $cle => $valeur) {
  echo "$cle : $valeur<br>";
}
```

c) `$a` est **indexé** : PHP numérote les cases 0, 1, 2 automatiquement. `$b` est **associatif** : les cases sont repérées par les clés `"x"` et `"y"`.